SOFTWARE ARCHITECTURE & SPECIFICATION REPORT

CampusConnect

Smart Event Management & Team-Matching Platform

An elegant, high-performance Next.js application built with React 19, Prisma ORM, and Neon PostgreSQL. CampusConnect gamifies student engagement through automated QR ticket check-ins, milestone achievement badges, AI-driven skill-complementary team matching, interactive support desk chats, and customized student resume portfolios. This document provides an in-depth technical walkthrough of the software's components, database design, API route workings, and complete deployment protocols.

TECHNICAL METADATA

Framework Stack	:	Next.js 16.2.9 (App Router) & React 19.2.4
Database Layer	:	Prisma ORM with Neon Serverless PostgreSQL
Styling System	:	Vanilla CSS Custom Variables & Theme Engine
Document Version	:	1.0.0 (Production Release Specification)
Date Generated	:	June 24, 2026
Classification	:	Confidential / System Development Documentation

Table of Contents

1. Executive Summary & Vision

CampusConnect is an integrated digital ecosystem designed for higher education institutions to manage extracurricular events, technical hackathons, workshops, and cultural celebrations. The project goes beyond traditional event listings by creating a dynamic, gamified portal that connects students based on skills and interest. By solving core campus pain points - such as manual attendance logs, friction in project team formation, and lack of profile visibility - CampusConnect acts as a unified platform representing a student's extracurricular lifecycle.

The platform operates on three pillars: automated event administration (using dynamic QR ticket check-ins), gamified progression (where check-ins and profile completions award points and unlock skill-based digital badges), and matching utilities (using skill profiles to match individuals into balanced project teams). This encourages continuous learning, peer cooperation, and active campus participation.

2. Architecture & Technologies Stack

CampusConnect uses a modern, robust, serverless architecture that guarantees rapid response times, flexible styling, and absolute database consistency. The following diagram illustrates the stack components:

Core Technical Stack Components

- **React 19 & Next.js 16** Uses Next.js App Router for server-side layouts and client-side page rendering. API Route Handlers act as the serverless backend, handling transactional operations.
- **Prisma ORM Client** Acts as the query builder and schema migrator. Enables type-safe database queries, cascading deletes, and complex table joins for notifications, skills, and invites.
- **Neon Serverless Postgres** Cloud-hosted PostgreSQL database featuring connection pooling and automatic scaling. Neon handles high-throughput queries during heavy event check-ins.
- **Dynamic Theme Engine** Implemented purely using Vanilla CSS variables (`global.css`). Themes (System default, Light, Dark, High-Contrast) and Accents (Blue, Green, Purple) are applied by mutating `data-theme` attributes on the root HTML document.
- **Client-side PDF Exporter** Integrates `html2canvas` and `jspdf` to convert modular HTML DOM elements into high-definition certificate documents download links.

3. Database Schema Specification (Prisma)

The relational database schema is configured in `prisma/schema.prisma`. It utilizes 13 interconnected models with enforced referential integrity and cascade options. Below is a breakdown of the models:

Model Name	Purpose & Scope	Key Relations
User	Profiles, point tallies, ranks, and theme preferences.	Skills, Badges, Certs, Regs
Skill	Technology, Design, Business catalog.	userSkills (Join Table)
UserSkill	Join table storing student level (1-100) per skill.	User, Skill
Badge	Achievements (Volunteer, Collaborator, etc.).	users (UserBadge join)
UserBadge	Earned achievements with timestamps.	User, Badge
Certificate	Credentials earned, matching verification URLs.	User
Event	Details on location, time, capacities, and costs.	registrations, teams
Registration	Enrolls students, logs check-ins, saves ticket QR.	User, Event
Team	Hackathon project groupings.	Event, members, invitations
TeamMember	Identifies team members and lead statuses.	Team, User
TeamInvitation	Manages pending, accepted, or declined requests.	Team, Sender, Recipient
Activity	Activity feeds capturing system actions.	User
SupportMessage	Student-helpdesk messaging threads.	User

Critical Schema Constraint: The `Registration` model enforces an `@@unique([userId, eventId])` constraint to prevent a student from registering twice for the same event. It also enforces a unique constraint on `qrCodeString` to secure attendance simulation.

4. Feature Workings & Internal Mechanisms

4.1 Authentication & Profile Logic

CampusConnect uses a cookie-based session tracker. Since the application runs within a secure campus intranet, authentication matches the user's name and email against database records.

1. Session Check: On page load, `DashboardLayout` fires a GET request to `/api/auth/session`. If a `userId` cookie exists, the handler queries the database, including the user's associated skills and badges. If no cookie is present, an overlays displays asking for Full Name and Email.

2. Login & Auto-Registration: When the student enters their details in the modal, a POST request is sent to `/api/auth/register`. If the user email is already registered, the backend logs them in by setting a persistent `userId` cookie. If the email is new, a student profile is automatically created with default fields (College of Engineering, CS Dept, Graduation 2027) and initialized with starting skills (React, Python). A welcome record is also logged in the `Activity` table.

4.2 Smart Event Registration & Ticketing

The event module manages registration, payment confirmations, ticket generation, and check-in simulations. The logic flow operates as follows:

1. Discovery: Students browse events filtered by category (Innovation, Cultural, Academic, Workshop, Career, Sports) and search queries. This makes a fetch call to `/api/events` which runs raw Prisma filters.

2. Registration & Validation: The student enters registration details (phone, place, year, department). If the event is paid, a transaction ID field is collected. The POST endpoint `/api/events/[id]/register` validates capacity, saves the registration record, and generates a unique ticket string matching: `CC-REG-{uuid}`.

4.3 Dynamic AI Team Matching Engine

To alleviate the stress of finding project partners during hackathons, CampusConnect implements a skill-based team matching engine.

1. Skill Definitions: Core teams have requirements predefined in a system record. E.g., 'Team Nexus' requires ['React', 'Python', 'UI/UX', 'ML']. 'Circuit Breakers' requires ['Arduino', 'C++', 'PCB Design', 'Robotics'].

2. Matching Algorithm: The matching endpoint `/api/teams/match` fetches the active user's skills and cross-references them with the requirements of each team. The matching score begins at a base of 50% and increments dynamically. For example, if matching with 'Team Nexus', the score increases by 10% for having React, 10% for Python, 7% for TypeScript, and 7% for Node.js. The final score is capped at a logical maximum of 98% to simulate room for improvement, and suggestions are returned sorted descending.

3. Networking Invites: Students can send join requests directly from the matching dashboard, which creates a PENDING record in `TeamInvitation`. The team leader can approve or decline the invitation. Accepting a request inserts a new `TeamMember` row and updates `openPositions` count.

4.4 Gamification System & Achievements

A cornerstone of the application is user motivation via points and unlockable badges, creating a self-sustaining cycle of participation.

1. Attendance Check-in Simulation: Real-world ticket scanners trigger `/api/events/[id]/check-in` when checking QR codes. The web dashboard embeds a ticket simulator where students can trigger this endpoint. When called, the backend:

- **Checks-in** Applies the date timestamp to the registration `attendedAt` column.
- **Awards Points** Increments the student's overall profile `points` count by 50 points.
- **Logs Activity** Creates a detailed `Activity` log tracking attendance verification.
- **Unlocks Milestones** Queries the database for total attended events by this student. It triggers badge unlocks:
- *** 'Volunteer' Badge** Unlocked upon completing the first (1st) event check-in.
- *** 'Collaborator' Badge** Unlocked upon completing two (2) event check-ins.
- *** 'Tech Enthusiast'** Unlocked upon completing five (5) event check-ins.

2. Leaderboards: The `/api/leaderboard` endpoint lists all students ordered by points descending. The UI displays overall score, department, badges count, total events attended, and a dynamic skill score.

4.5 Interactive Support Helpdesk

To handle queries, the application includes a dedicated support desk chat. Students can send query messages to campus administration through the help panel.

The messaging backend `/api/support` reads and writes records to the `SupportMessage` database table. Messages are tagged with the sender's identity ('STUDENT' or 'COLLEGE'). When a student navigates to the Help page, the portal fetches the historical chat thread associated with their specific `userId`. New submissions execute a POST that instantly updates the database, allowing organizers to respond through administrative consoles.

5. Local Installation & Development Setup

Follow these steps to set up and run the CampusConnect project locally on your machine:

- **Step 1: Clone Repository** Extract the project folders and enter the root directory: cd campusconnect
- **Step 2: Install Node modules** Run 'npm install' to fetch dependencies (React 19, Next.js 16, Prisma client, jspdf, html2canvas).
- **Step 3: Setup Env Variables** Create a '.env' file in the root directory. Add your Neon PostgreSQL database connection string: DATABASE_URL="postgresql://..."
- **Step 4: Push DB Schema** Generate Prisma Client and push tables to Neon database by executing: 'npx prisma db push'
- **Step 5: Seed Database** Populate skills, default badges, and demo student users: 'npx prisma db seed'
- **Step 6: Start Dev Server** Start the local Next.js development server: 'npm run dev'

Once the server starts, open http://localhost:3000 in your browser. You can click 'Get Started' and log in with the seeded user 'alex.chen@campusconnect.edu' to inspect pre-seeded data, certifications, and matches.

6. Production Deployment Protocol

To host CampusConnect in a production environment, follow this structured deployment protocol utilizing Vercel and Neon PostgreSQL:

6.1 Database Deployment on Neon PostgreSQL

1. Create a free-tier project on Neon (https://neon.tech) and select a PostgreSQL version compatible with Prisma.

2. Copy the connection string from the dashboard. Enable the 'pooled' connection variant if deploying on serverless architectures.

3. Locally, run the Prisma migration script or push command using the database URL to verify connection and provision database tables.

6.2 Application Deployment on Vercel

1. Import the CampusConnect GitHub repository into your Vercel team dashboard.

2. In the project Build Settings, use default commands: Build command: 'next build', Output directory: '.next'.

3. Under 'Environment Variables', add: 'DATABASE_URL' matching the Neon database string.

4. Click Deploy. Vercel automatically compiles the static pages, builds serverless endpoint routes, and serves the site globally.

6.3 Verifying the Production Build

After deployment finishes, execute these checks:

- **Domain check** Ensure page redirects correctly from the primary vercel.app domain.
- **Auth persistence** Verify cookie headers are saved securely during login/register.
- **Prisma execution** Test event booking and check-in to confirm data is written to Postgres.
- **PDF Cert Download** Inspect certificate generation; confirm PDF downloads successfully.

END OF SPECIFICATION DOCUMENT

For further queries or source integrations, contact: admin@campusconnect.edu